Software Architecture and Design Specification

Project: Student Record System

Version 1.0

Authors: Team 7

Upadrasta Naga Sarvani (PES2UG24CS567)
Vinaayak G Dasika (PES2UG24CS588)
Yaseen Mulla (PES2UG24CS607)
Eshwar B H (PES2UG25CS808)

Date: 14/09/2026

Status: Draft

Revision History

Version	Date	Author	Change Summary
1.0	14/09/2026	Team 7	Initial draft of SAD, derived from SRS v1.0

Approvals

Role	Name	Signature / Date
Course Coordinator / Instructor	Pavithra Ma’am	

1. Introduction

1.1 Purpose

This document specifies the architecture and design of the Student Record System.

1.2 Scope

Covers login and role-based access, student record management, attendance and marks, and reporting/backup for Student, Faculty and Admin roles.

1.3 Audience

Developers, course instructor/evaluator, and maintenance team (Team 7).

1.4 Definitions

RBAC, GPA, UI, API, JWT, TLS/HTTPS, RTM.

2. Document Overview

2.1 How to use this document

Provides architectural deliverables including UML diagrams, component descriptions, API design, and a traceability matrix.

2.2 Related Documents

SRS, RTM.

3. Architecture

3.1 Goals & Constraints

Goals: role-based access, reliable data persistence, daily backup, usable on laptop and phone. Constraints: single-college deployment, browser-only clients, no special hardware.

3.2 Stakeholders & Concerns

Students: privacy, usability Faculty: quick attendance/marks entry Admin: accurate records, reporting Developers: modularity

3.3 Component (UML) Diagram

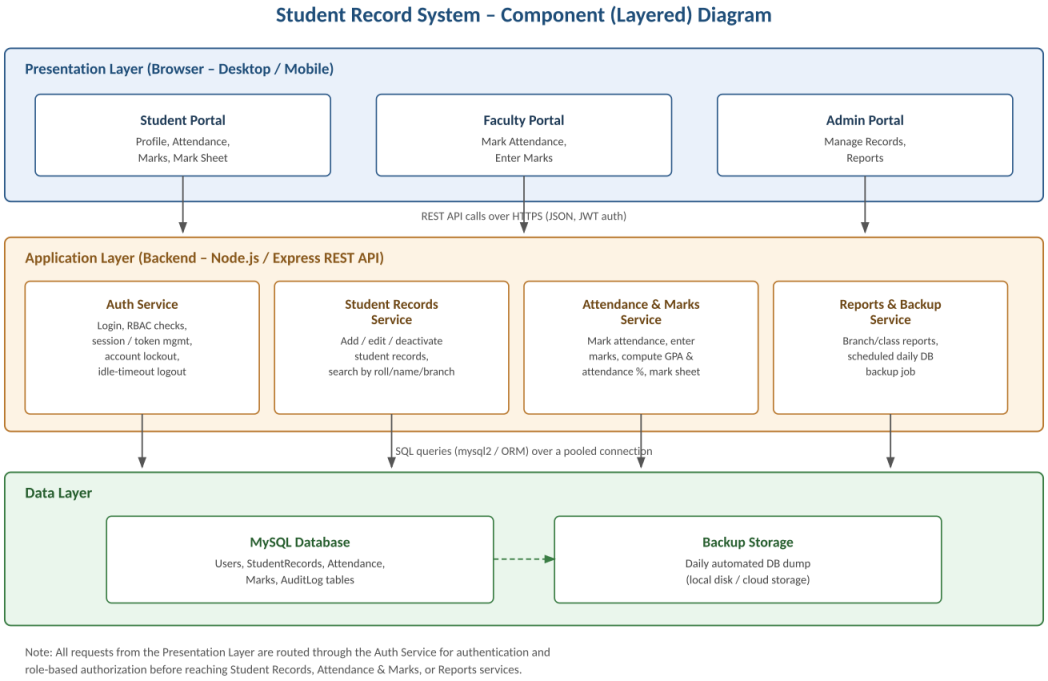


Figure 1: Layered component diagram of the Student Record System

3.4 Component Descriptions

- Student/Faculty/Admin Portals: role-specific browser UI.
- Auth Service: handles login, password hashing, account lockout, session timeout, role validation.
- Student Records Service: add/edit/deactivate/search student records.
- Attendance & Marks Service: attendance marking, marks entry, GPA calculation, mark sheet generation.
- Reports & Backup Service: branch/class reports, daily database backup.
- Database: stores student, attendance, marks and audit data.

3.5 Chosen Architecture Pattern and Rationale

Layered architecture chosen for clear separation of concerns and simplicity at this project's scale. Microservices rejected as overly complex for a single-college system.

3.6 Technology Stack & Data Stores

Node.js, Express REST API, MySQL DB, TLS encryption. Browser-based frontend, responsive for desktop and mobile.

3.7 Risks & Mitigations

Risk: Unauthorized role access via URL guessing → Mitigation: server-side role check on every endpoint.

Risk: Data loss on server restart → Mitigation: all writes persisted to MySQL immediately.

Risk: Accidental record loss → Mitigation: deactivate instead of delete.

Risk: Database loss → Mitigation: automated daily backup.

3.8 Traceability to Requirements

SRSF-001 (Role-based login) → Auth Service

SRSF-010–014 (Record management, search) → Student Records Service

SRSF-020–023 (Marks, attendance, GPA) → Attendance & Marks Service

SRSF-030, SRSF-040 (Reports, backup) → Reports & Backup Service

SRSSR-001–005 (Security requirements) → Auth Service (cross-cutting)

3.9 Security Architecture

Threat Modeling (STRIDE): Spoofing → hashed passwords; Tampering → server-side validation; Info disclosure → TLS; DoS → login lockout; Elevation of Privilege → RBAC.

4. Design

4.1 Design Overview

The system is designed as four backend modules behind a single REST API, called by three role-specific frontend views. Every request outside of login carries a JWT; each service independently checks that the token's role is allowed to perform the requested action, per SRSSR-003. The two sequence diagrams below walk through the most representative Student and Faculty flows.

4.2 UML Sequence Diagrams

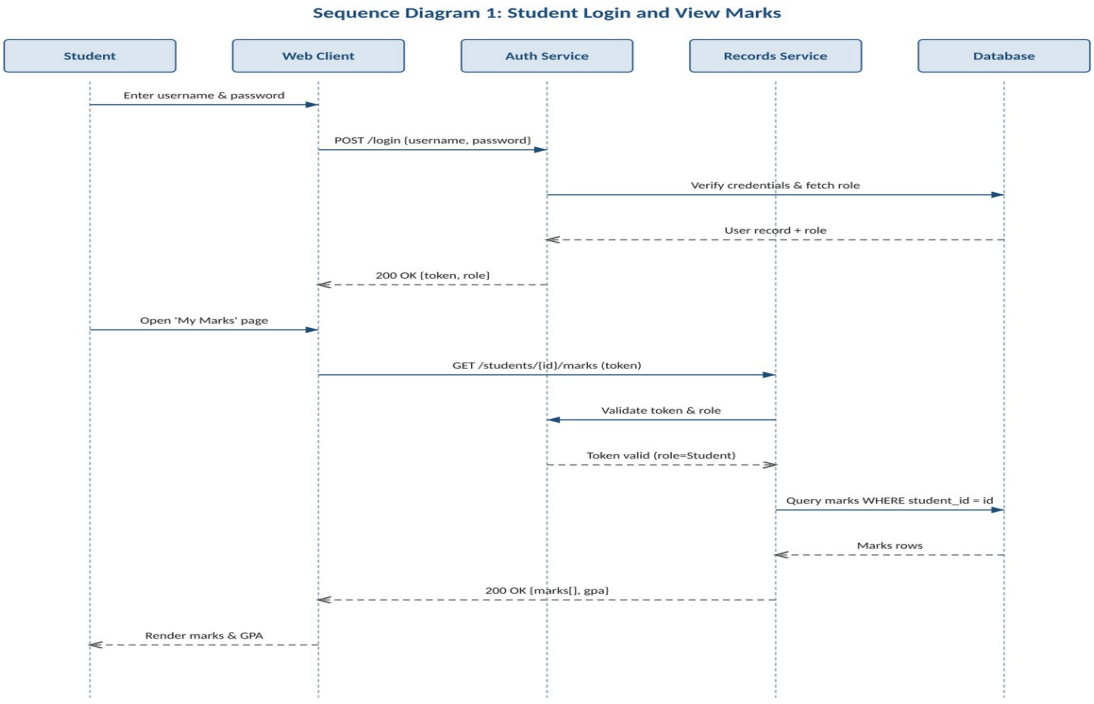


Figure 2: Sequence diagram – Student login and view marks

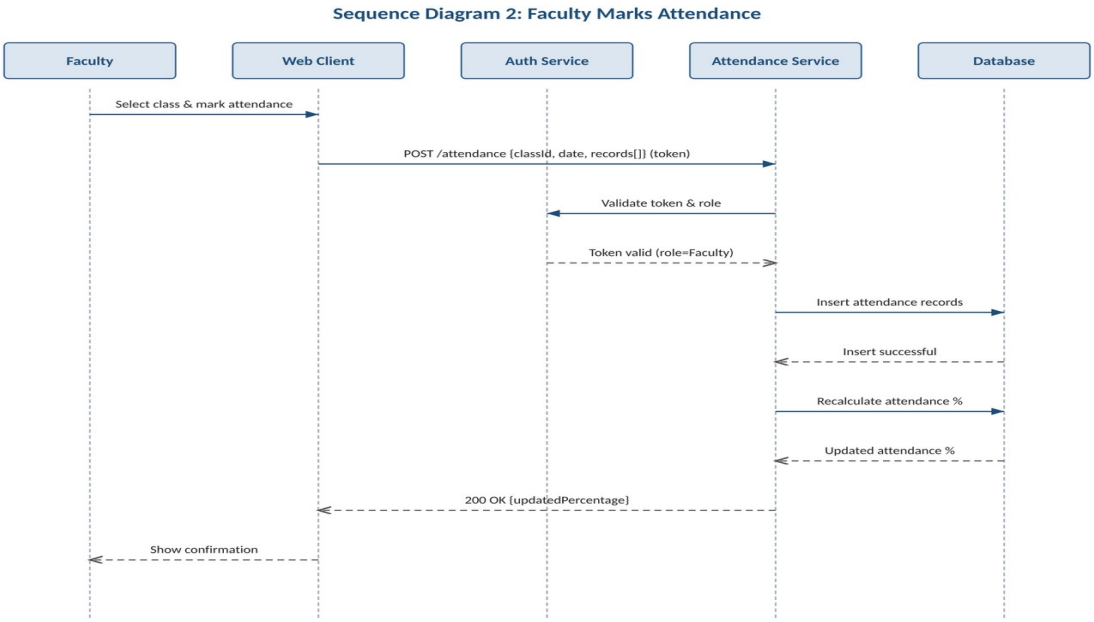


Figure 3: Sequence diagram – Faculty marks attendance

4.3 API Design

Interface definitions for two representative components (Auth Service and Attendance & Marks Service):

Auth Service

- Endpoint: /login
- Method: POST
- Request: { username, password }
- Response: { status, token, role }
- Errors: 401 Invalid credentials, 423 Account locked (after 3 failed attempts)

Attendance & Marks Service

- Endpoint: /attendance
- Method: POST
- Request: { classId, date, records: [{ studentId, present }] } (Authorization: Bearer <token>)
- Response: { status, updatedPercentage }
- Errors: 401 Invalid/expired token, 403 Role is not Faculty, 400 Malformed attendance payload

4.4 Error Handling, Logging & Monitoring

Standardized JSON error responses across all services: { status, errorCode, message }. No sensitive information (passwords, tokens) is ever written to logs. Monitoring tracks login failure rate (to catch lockout/brute-force patterns), API error rate, and basic uptime, so the team can confirm SRS-NF-002 (the system stays usable through a full class demo/session) is being met.

4.5 UX Design

Each role gets a dedicated dashboard rather than one generic screen: Student → profile/attendance/marks; Faculty → attendance/marks entry; Admin → record management and reports. The interface follows SRS-NF-005: usable on both a laptop and a phone browser without major layout issues, using a responsive layout rather than separate mobile/desktop codebases.

4.6 Open Issues & Next Steps

- Confirm final choice of ORM vs. raw SQL (mysql2) for the Data Layer
- Decide exact JWT expiry / idle-timeout duration (SRS suggests ~15 min)
- Design the AuditLog table schema in detail (fields, retention)
- Finalize backup storage location (local disk vs. cloud) for SRSF-040

5. Appendices

5.1 Glossary

RBAC, GPA, UI, API, JWT, TLS/HTTPS, RTM, ORM — see Section 1.4 for definitions.

5.2 References

- Software Requirements Specification – Student Record System, v1.0 (Team 7)
- IEEE 42010 – Systems and software engineering — Architecture description
- OWASP – web application security guidance (used for the threat model in §3.9)

5.3 Tools

- Diagrams: generated programmatically (SVG) for the component and sequence diagrams in this document
- Suggested for future editing: draw.io / PlantUML
- API testing: Postman or similar, once endpoints are implemented